Rislyn Raja
CSE 4820: Introduction to Machine Learning
Professor Ellie
May 8, 2025

Final Project - Data Analysis

1. Data Information

Before using the data, I started by preprocessing it. First, I downloaded the csv files from the website: https://www.kaggle.com/datasets/arashnic/hr-analytics-job-change-of-data-scientists/data. This data shows general information on data scientists like city, gender, education level, years of experience, company size and more as well as a 'target' value of either 1.0 or 0.0 on whether or not they switched to a new job. Two sets of data were given on the website: training data and testing data however I later split the training data into test, validate, and split because the testing data did not come with a target column. After downloading, I ran this line to find the columns with empty values: print(df.isnull().sum()). I dropped any duplicate rows and then I was able to determine that there were 8 columns that had missing values.

For the columns: gender with 4508 missing values I replaced with "Unknown", enrolled_university with 386 missing values I replaced with the mode, education_level with 460 missing values I replaced with the mode, major_discipline with 2813 missing values I replaced with "Unknown", experience with 65 missing values I replaced with "Unknown", company_size with 5938 missing values I replaced with "Unknown", company_type with 6140 missing values I replaced with "Unknown", and last_new_job with 423 missing values I replaced with mode.

After this, I dropped the enrolee_id column from the training data set since it was not needed. The next step of preprocessing included splitting the data into train/validate/test. I used a 60-20-20 split since it is generally considered a good split for most data sets. The code for this section looked like:

```
df = pd.read_csv("/workspaces/4820/project/aug_train.csv") #
first, loading data

df.drop_duplicates(subset='enrollee_id', keep='first',
inplace=True) # drop duplicates

# filling in missing data
df['gender'].fillna('Unknown')
df['enrolled_university'].fillna(df['enrolled_university'].mode
()[0])
```

```
df['education_level'].fillna(df['education_level'].mode()[0])
df['major_discipline'].fillna('Unknown')
df['company_size'].fillna('Unknown')
df['company_type'].fillna('Unknown')
df['last_new_job'].fillna(df['last_new_job'].mode()[0])
df['experience'].fillna('Unknown')

ids = df['enrollee_id'] # save for later

# cleaning data by removing unnecessary columns
df.drop(['enrollee_id'], axis=1, inplace=True) # not needed for
training

catcols = df.select_dtypes(include='object').columns  # get
categorical columns

for col in catcols:
     le = LabelEncoder()
     df[col] = le.fit_transform(df[col].astype(str))  # convert
to string just in case

X = df.drop("target", axis=1) # features = X, target = y
y = df["target"].astype(int)  # currently either 1.0 or 0.0 so
converting to int

X_trainval, X_test, y_trainval, y_test = train_test_split(X, y,
test_size=0.2, random_state=42) # 60/20/20 train, validate,
test split

X_train, X_val, y_train, y_val = train_test_split(X_trainval,
y_trainval, test_size=0.25, random_state=42)
```

2. Approaches

Among all of the approaches, there was some code that was repeated and only slightly modified to fit each model's specific variables. This common code contains the functionality necessary for printing out valuable data that will later be used for determining which model is the best. It will print out accuracy, F1 score, the best parameters, etc, first for the validation data set and then the test data set. Afterwards, it will also use matplot to display the accuracy graph. This code looks like:

```
# validation set
valpreds = rfbestmodel.predict(X_val)
```

```
print("Random Forest Classifier - Validation Results")
print("Best Params:", rfgrid.best_params_)
print("Validation Accuracy:", accuracy_score(y_val, valpreds))
print("Validation F1 Score:", f1_score(y_val, valpreds))
print("Confusion Matrix:\n", confusion_matrix(y_val, valpreds))
print("Classification Report:\n", classification_report(y_val,
valpreds))

# test set
testpreds = rfbestmodel.predict(X_test)
print("\nRandom Forest Classifier - Test Results")
print("Test Accuracy:", accuracy_score(y_test, testpreds))
print("Test F1 Score:", f1_score(y_test, testpreds))
print("Confusion Matrix:\n", confusion_matrix(y_test,
testpreds))
print("Classification Report:\n", classification_report(y_test,
testpreds))

# output test predictions to csv file
enrolleeids = ids.loc[X_test.index]
output = pd.DataFrame({
   'enrollee_id': enrolleeids,
   'target': testpreds
})
output.to_csv("randomforestresults.csv", index=False)

# learning curve for the best model
trainsizes, trainscores, valscores = learning_curve(
   estimator=rfbestmodel,
   X=X_train,
   y=y_train,
```

```
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)

# Calculate means and stds
trainmean = np.mean(trainscores, axis=1)
trainstd = np.std(trainscores, axis=1)
valmean = np.mean(valscores, axis=1)
valstd = np.std(valscores, axis=1)

# graphing learning curve
plt.figure(figsize=(8, 6))
plt.plot(trainsizes, trainmean, color='darkorange', marker='o',
label='Training accuracy')
plt.fill_between(trainsizes, trainmean - trainstd, trainmean +
trainstd, alpha=0.15, color='darkorange')
plt.plot(trainsizes, valmean, color='navy', linestyle='--',
marker='s', label='Validation accuracy')
plt.fill_between(trainsizes, valmean - valstd, valmean +
valstd, alpha=0.15, color='navy')
plt.grid()
plt.xlabel('Number of training examples')
plt.ylabel('Accuracy')
plt.title('Random Forest Learning Curve')
plt.legend(loc='lower right')
plt.ylim([0.5, 1.03])
plt.tight_layout()
plt.show()
```

Approach 1 - Random Forest Classification

The first approach I selected was Random Forest Classification which is an ensemble classification method imported using sklearn. In most cases, it is used to reduce overfitting and can help increase the accuracy of the model. The code for the Random Forest Classification model looks like:

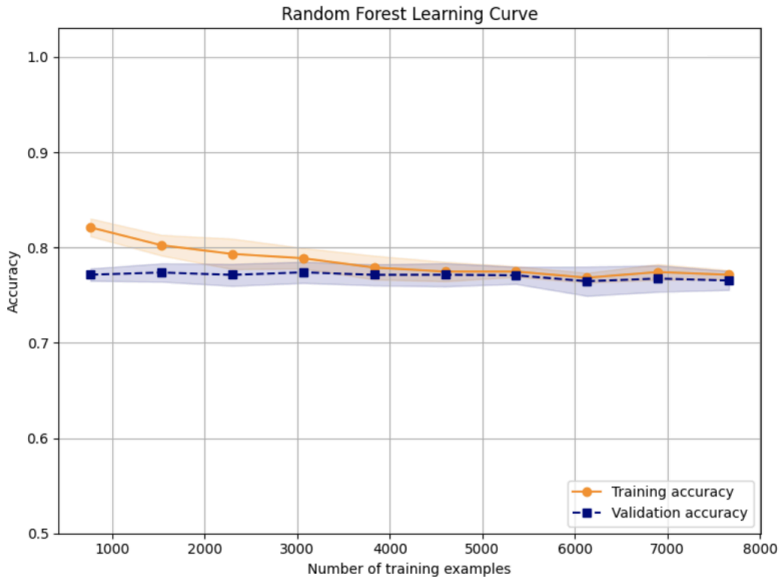
```
rfparams = {
    'n_estimators': [10, 20],
    'max_depth': [1, 2, 5],
    'min_samples_split': [2, 3, 4],
    'min_samples_leaf': [2, 3, 4]
}

rf = RandomForestClassifier(class_weight='balanced',
random_state=42)
rfgrid = GridSearchCV(rf, rfparams, cv=3, scoring='accuracy',
n_jobs=-1)
rfgrid.fit(X_train, y_train)

rfbestmodel = rfgrid.best_estimator_
```

Additionally, the results from the code for Random Forest Classification can be seen in the images below. The validation accuracy was around 0.77 and test accuracy was a similar number close to 0.77. The validation F1 score was around 0.62 and the test F1 score was around 0.64.

```
Random Forest Classifier – Validation Results
Best Params: {'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 20}
Validation Accuracy: 0.7659185803757829
Validation F1 Score: 0.6165027789653699
Confusion Matrix:
 [[2214  641]
 [ 256  721]]
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.78      0.83      2855
           1       0.53      0.74      0.62       977

    accuracy                           0.77      3832
   macro avg       0.71      0.76      0.72      3832
weighted avg       0.80      0.77      0.78      3832


Random Forest Classifier – Test Results
Test Accuracy: 0.7742693110647182
Test F1 Score: 0.6354825115887063
Confusion Matrix:
 [[2213  664]
 [ 201  754]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.77      0.84      2877
           1       0.53      0.79      0.64       955

    accuracy                           0.77      3832
   macro avg       0.72      0.78      0.74      3832
weighted avg       0.82      0.77      0.79      3832
```

The graph of the accuracy for both training and validation for Random Forest Classification can be seen below. The training accuracy gets lower for the number of training samples so it shows that overfitting is not occurring. Both accuracy values are converging around 0.77.



Approach 2 - Gradient Boosting Classification

Gradient Boosting Classification, like Random Forest, is also from sklearn and an ensemble classification model. For this reason, the same parameters were used for both.

Just like before, GridSearchCV was used to determine the best parameters. The code for this model looks like:

```
gbparams = {
    'n_estimators': [10, 20],
    'max_depth': [1, 2, 5],
    'min_samples_split': [2, 3, 4],
    'min_samples_leaf': [2, 3, 4]
}

gb = GradientBoostingClassifier(random_state=42)
gbgrid = GridSearchCV(gb, gbparams, cv=3, scoring='accuracy',
n_jobs=-1)
gbgrid.fit(X_train, y_train)

gbbestmodel = gbgrid.best_estimator_
```

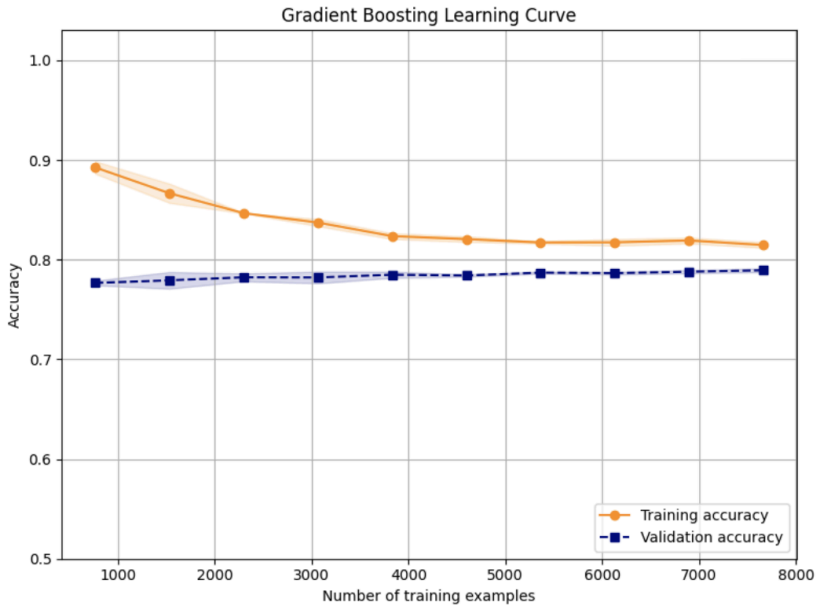
Additionally, the results from this code for Gradient Boosting Classification can be seen below. The accuracy was higher with around 0.79 for validation and around 0.8 for test. However, the F1 score was lower with around 0.57 for validation and a very low 0.26 for test. Since F1 score is considered a more important metric for imbalanced data, this was something I strongly took into consideration later when choosing the best model.

```
Gradient Boosting Classifier - Validation Results
Best Params: {'max_depth': 5, 'min_samples_leaf': 3, 'min_samples_split': 2, 'n_estimators': 20}
Validation Accuracy: 0.7914926931106472
Validation F1 Score: 0.5738666666666666
Confusion Matrix:
 [[2495  360]
 [ 439  538]]
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.87      0.86      2855
           1       0.60      0.55      0.57       977

    accuracy                           0.79      3832
   macro avg       0.72      0.71      0.72      3832
weighted avg       0.79      0.79      0.79      3832


Gradient Boosting Classifier - Test Results
Test Accuracy: 0.798277661795407
Test F1 Score: 0.2576943140323422
Confusion Matrix:
 [[2498  379]
 [ 394  561]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.87      0.87      2877
           1       0.60      0.59      0.59       955

    accuracy                           0.80      3832
   macro avg       0.73      0.73      0.73      3832
weighted avg       0.80      0.80      0.80      3832
```

Lastly, the graph of the accuracy for both training and validation for Gradient Boosting Classification can also be seen below. Similar to the Random Forest

Classification graph, both lines seem to be converging and there is no overfitting occurring.



Approach 3 - SVM

The third approach I used was SVM. Standing for Support Vector Machine, it is a simple method that is also imported using sklearn like the two former approaches that were used. Its specialty is that it can be kernelized (where the data input is replaced with smaller input) if needed to be used for linear problems. The code for this model looks like:

```
svmparams = [
   {'svc__kernel': ['linear'], 'svc__C': [0.1, 1, 10, 30]},
   {'svc__kernel': ['poly'], 'svc__C': [0.1, 1, 10],
'svc__degree': [2, 3, 4], 'svc__gamma': ['scale', 0.1],
'svc__coef0': [0, 0.5]},
   {'svc__kernel': ['rbf'], 'svc__C': [0.1, 1, 10], 'svc__gamma':
['scale', 0.1, 1]},
   {'svc__kernel': ['sigmoid'], 'svc__C': [0.1, 1, 10],
'svc__gamma': ['scale', 0.1], 'svc__coef0': [0, 1]}
]

svm = make_pipeline(StandardScaler(),
SVC(class_weight='balanced', probability=True, random_state=42))
svmgrid = GridSearchCV(svm, svmparams, cv=3, scoring='accuracy',
n_jobs=-1)
svmgrid.fit(X_train[:2000], y_train[:2000])  # limit for faster
results
```

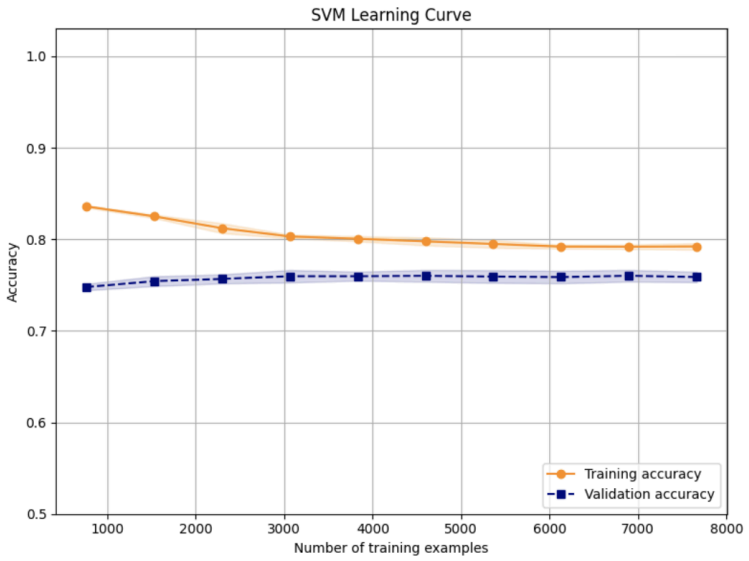
```
svmbestmodel = svmgrid.best_estimator_
```

The results from this code can be seen in the images below. The validation accuracy was 0.75 and the test accuracy was 0.76 which was very similar. The validation F1 score was 0.47 and the test F1 score was 0.49.

```
SVM - Validation Results
Best Params: {'svc__C': 0.1, 'svc__coef0': 0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel': 'poly'}
Validation Accuracy: 0.7510438413361169
Validation F1 Score: 0.47
Confusion Matrix:
 [[2455  400]
 [ 554  423]]
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.86      0.84      2855
           1       0.51      0.43      0.47       977

    accuracy                           0.75      3832
   macro avg       0.66      0.65      0.65      3832
weighted avg       0.74      0.75      0.74      3832


SVM - Test Results
Test Accuracy: 0.7578288100208769
Test F1 Score: 0.4940021810250818
Confusion Matrix:
 [[2451  426]
 [ 502  453]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.85      0.84      2877
           1       0.52      0.47      0.49       955

    accuracy                           0.76      3832
   macro avg       0.67      0.66      0.67      3832
weighted avg       0.75      0.76      0.75      3832
```

The graph of the accuracy for both training and validation for SVM can also be seen below. Similar to the previous two approaches, the lines seem to be converging and there is no overfitting since the training accuracy is getting lower and then plateauing after a certain amount of examples.



Approach 4 - Balanced Bagging Classifier

The last approach I used was Balanced Bagging Classifier. Unlike the last three approaches, this approach was not from sklearn but from imblearn. It balances data

during the training process which makes it the perfect choice for imbalanced data sets. The code for this model looks like:

```
bbcparams = [
    {
        'n_estimators': [10, 20, 50],
        'max_samples': [0.5, 0.7, 1.0],
        'estimator__max_depth': [None, 10, 20],
        'estimator__min_samples_split': [2, 5, 10],
        'estimator__min_samples_leaf': [1, 2, 4]
    }
]

bbc =
BalancedBaggingClassifier(estimator=DecisionTreeClassifier(),
random_state=42, n_jobs=-1) # temp
bbcgrid = GridSearchCV(bbc, bbcparams, cv=3, scoring='accuracy',
n_jobs=-1) # using the params
bbcgrid.fit(X_train, y_train)

bbcbestmodel = bbcgrid.best_estimator_
```

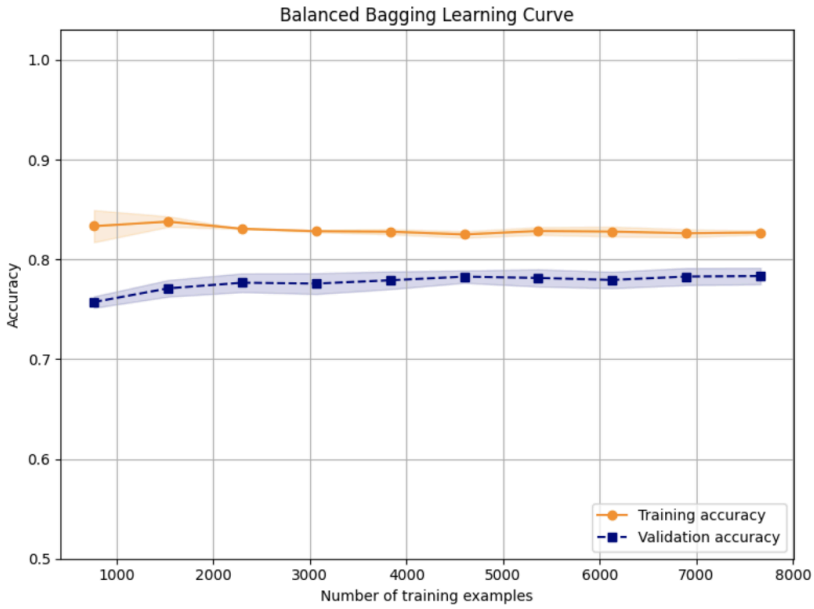
Additionally, the results from the code for Balanced Bagging Classifier can be seen in the images below. The validation accuracy was 0.78 and the test accuracy was around 0.79 which was just slightly more than Random Forest Classifier's accuracy. The F1 score for validation was around 0.63 and around 0.64 for test. This was also very similar to Random Forest Classifier's F1 score.

```
Balanced Bagging Classifier – Validation Results
Best Params: {'estimator__max_depth': 20, 'estimator__min_samples_leaf': 4, 'estimator__min_samples_split': 2, 'max_samples': 0.7, 'n_estimators': 50}
Validation Accuracy: 0.7831419624217119
Validation F1 Score: 0.6298440979955456
Confusion Matrix:
 [[2294  561]
 [ 270  707]]
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.80      0.85      2855
           1       0.56      0.72      0.63       977

    accuracy                           0.78      3832
   macro avg       0.73      0.76      0.74      3832
weighted avg       0.81      0.78      0.79      3832


Balanced Bagging Classifier – Test Results
Test Accuracy: 0.7875782881002088
Test F1 Score: 0.641409691629956
Confusion Matrix:
 [[2290  587]
 [ 227  728]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.80      0.85      2877
           1       0.55      0.76      0.64       955

    accuracy                           0.79      3832
   macro avg       0.73      0.78      0.75      3832
weighted avg       0.82      0.79      0.80      3832
```

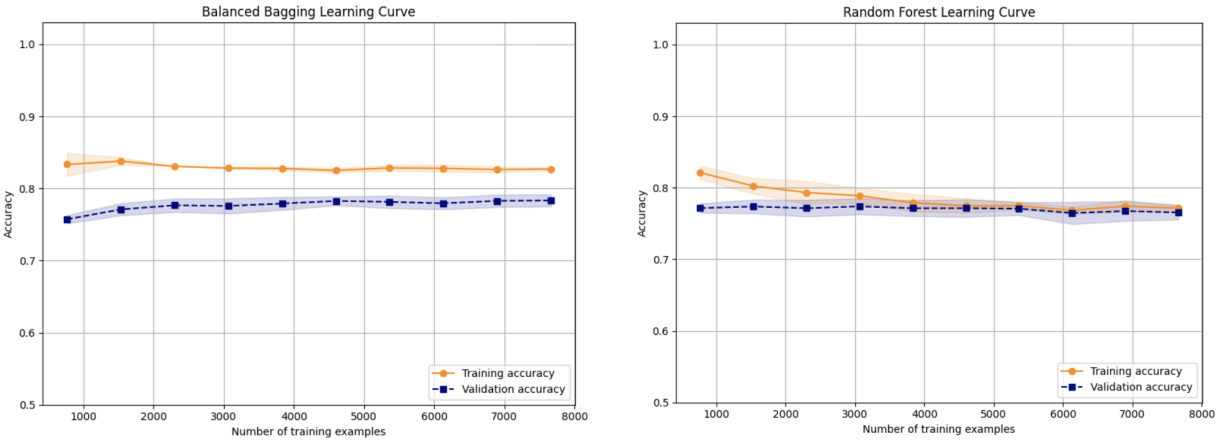
The graph of the accuracy for both training and validation for the Balanced Bagging Classifier can also be seen below. Unlike previous graphs, this one contains lines that have less variation as the number of training examples increases. Similar to

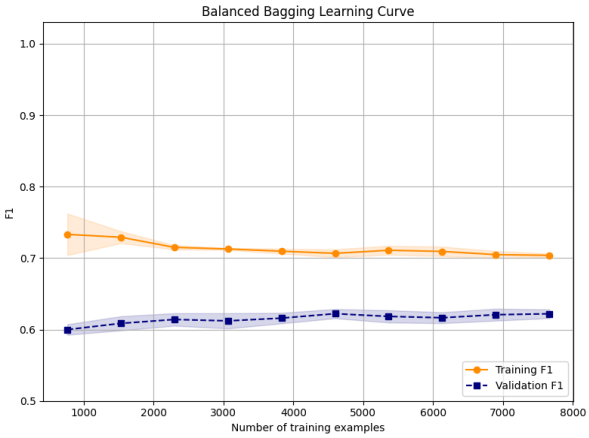
previous graphs, however, this one also does not show any evidence of overfitting and may only show underfitting.

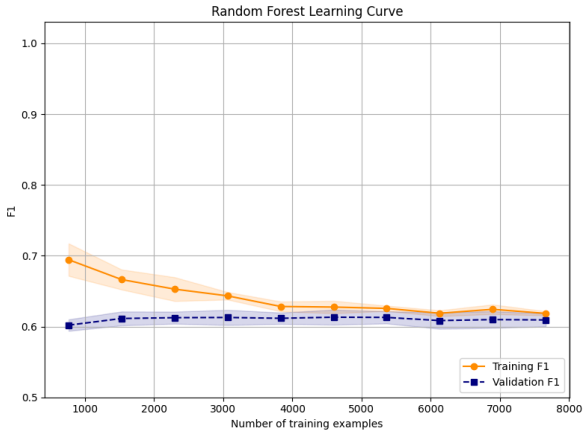


3. Model Selection

When selecting the model to use, I mostly focused on the F1 score since this dataset was imbalanced. I also took the accuracy into consideration and the learning curves to see whether or not the models contained overfitting or underfitting. Overall, the best model for this data was the Balanced Bagging Classifier. It had slightly better scores than the second best model: Random Forest. The learning curve was also slightly better since it did not show any major increase or decrease for the training accuracy as the number of training samples increased. The selected model's (Balanced Bagging) learning curve can be seen in the image below beside the learning curve for Random Forest. The first set of graphs is based on accuracy and the second set is based on the F1 score.

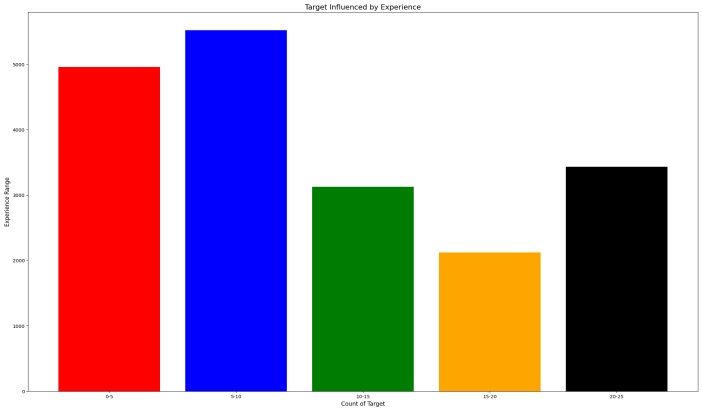






4. Previous Work

Most of the previous work I found online for this data set focused on data preprocessing methods. A few of the links I looked at were: https://www.kaggle.com/code/nkitgupta/advance-data-preprocessing and https://www.kaggle.com/code/rajagrawal7089/hr-analysis. The latter mostly focused on showing the effect that each feature (for example: experience, gender, etc.) had on the 'target' value. A graph from the second source can be seen below:



5. My Contribution

I chose a slightly different method during pre-processing than I saw online. In previous examples, most opted for using the 'mode' for missing values but, I mostly opted for using a new value of 'Unknown' instead. I applied four models to the data and found results for each involving the accuracy score, F1 score, confusion matrix, and classification report, and more. The learning curve was also graphed for each one based on accuracy and later based on F1 score for the top two best approaches. I was able to determine that the best approach for this data set was Balanced Bagging Classifier from imblearn. Overall, I found that most of the models seemed to be underfit rather than overfit and may succeed if given more features during model training. Initially, while

working on this project, I had fewer features for each model but, since the models appeared very underfit, I added more which was able to improve them. If possible in the future, I think it would be interesting to see how other imblearn models perform with this data set as well.